

University of Rostock

Institute of Computer Science

SQL Injection Demonstration Project

Technical Documentation

Cybersecurity Course Project

Group C

Team Members: Elshan Tahermanesh
Ashwin Kumar Selvavinayagam
Matthias Losch
Muska Ijaz
Surbhi Mehto
Pratham Tatraiya
Pauline Meuer

Instructor: Prof. Dr. Clemens Cap

Project Type: Educational Cybersecurity Laboratory

Technology Stack: React, Flask, SQLite, Docker

Declaration of Educational Purpose

This project was created exclusively for teaching, demonstration, and controlled laboratory use. It illustrates how SQL injection vulnerabilities can arise, how they can be recognized, and how secure implementations prevent them. The attack modules intentionally restrict execution to loopback or private network targets. Public Internet scanning is disabled. The destructive `DROP TABLE` demonstration is implemented as a non-destructive capability check and does not transmit or execute the destructive statement.

Abstract

The `SQL_INJECT` project is an interactive web-based laboratory for demonstrating common SQL injection techniques. It combines a React single-page application with a Flask backend and a small SQLite database. The frontend provides instructional pages, a payload library, attack configuration forms, animated terminal output, result summaries, and local test logs. The backend exposes health, authentication, demonstration, and attack-execution endpoints. Six attack categories are represented: classic tautology injection, login bypass, UNION-based extraction, Boolean-based blind injection, SQL comment injection, and a safe `DROP TABLE` vulnerability demonstration.

The system is designed around a separate intentionally vulnerable target application, referred to in the project as the Golimar Store. The attack service sends controlled HTTP requests to this local target, analyzes the responses, and returns structured findings to the frontend. Safety controls include private-address validation, Docker-aware loopback translation, request timeouts, and non-destructive handling of destructive payloads. This document describes the motivation, requirements, architecture, implementation, attack scenarios, secure coding principles, deployment process, testing strategy, limitations, and future work of the project.

Keywords: SQL injection, web security, React, Flask, SQLite, Docker, cybersecurity education, secure coding.

Contents

Declaration of Educational Purpose	i
Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Project Motivation	1
1.3 Project Objectives	1
1.4 Scope	2
2 Requirements Analysis	2
2.1 Functional Requirements	2
2.2 Non-Functional Requirements	3
3 Technology Stack	4
4 System Architecture	4
4.1 Architectural Overview	4
4.2 Frontend Architecture	5
4.3 Backend Architecture	6
4.4 Database Layer	6
4.5 Docker Networking	7
5 Project Structure	7
5.1 Frontend Components	8
5.2 Backend Modules	9
6 API Design	9
6.1 General Endpoints	9
6.2 Attack Endpoints	9

6.3	Request Format	10
7	User Interface	10
7.1	Home Dashboard	10
7.2	Instructions	11
7.3	Attack Overview	12
7.4	Payload Library	13
7.5	Terminal Output	14
8	Attack Modules	15
8.1	Common Attack Workflow	15
8.2	Classic SQL Injection	16
8.2.1	Concept	16
8.2.2	Project Implementation	16
8.2.3	Security Impact	16
8.3	Login Bypass	16
8.3.1	Concept	16
8.3.2	Demonstration Target	17
8.4	UNION-Based SQL Injection	17
8.4.1	Concept	18
8.4.2	Table Enumeration	18
8.4.3	Demonstration Target	18
8.5	Boolean-Based Blind SQL Injection	18
8.5.1	Concept	18
8.5.2	Demonstration Target	19
8.6	SQL Comment Attack	20
8.6.1	Concept	20
8.6.2	Demonstration Target	20
8.7	DROP TABLE Demonstration	21
8.7.1	Concept	21

8.7.2	Non-Destructive Safety Design	21
8.7.3	Demonstration Target	22
9	Safety and Ethical Controls	23
9.1	Private-Network Restriction	23
9.2	Controlled Connectivity	23
9.3	Destructive Payload Handling	23
9.4	Data Handling	23
10	Secure Implementation Principles	23
10.1	Parameterized Queries	23
10.2	Password Security	24
10.3	Least Privilege	24
10.4	Input Validation	24
10.5	Error Handling	24
10.6	Security Comparison	25
11	Installation and Execution	25
11.1	Prerequisites	25
11.2	Docker Compose Execution	25
11.3	Manual Backend Execution	26
11.4	Manual Frontend Execution	26
11.5	Database Initialization	26
12	Demonstration Procedure	26
12.1	Example: Login Bypass Presentation	27
13	Testing Strategy	27
13.1	Backend Unit and Integration Tests	27
13.2	Frontend Tests	28
13.3	Manual Acceptance Test Matrix	28

14 Known Limitations	29
15 Recommended Improvements	30
16 Team Organization	31
17 Conclusion	31
A Selected Configuration Files	32
A.1 Docker Compose Configuration	32
A.2 Flask Application Factory	33
B Payload Reference	33
C References	34

List of Figures

1	Logical architecture of the SQL injection laboratory	5
2	Home dashboard of the SQL injection laboratory	11
3	Instructions overview	12
4	Attack module overview	13
5	Payload library	14
6	Terminal-style scanner output	15
7	Remote login bypass module	17
8	Boolean-based blind SQL injection module	19
9	SQL comment attack module	21
10	Non-destructive DROP TABLE vulnerability demonstration	22

List of Tables

1	Functional requirements	2
2	Non-functional requirements	3
3	Main technologies used in the project	4
4	Important frontend routes	5
5	General backend API endpoints	9
6	Attack execution API endpoints	9
7	Vulnerable and secure query construction	25
8	Manual acceptance test matrix	29
9	Project responsibility areas	31
10	Payloads included in the interactive presets	33

1 Introduction

1.1 Background

SQL injection is a class of application-layer vulnerability in which untrusted input is interpreted as part of an SQL command. The vulnerability normally appears when an application constructs a database query by concatenating user-controlled values directly into the SQL statement. An attacker can then alter the intended meaning of the query, bypass access controls, disclose records, modify data, or, in severe cases, execute destructive database operations.

Although parameterized queries are a well-established defense, SQL injection remains important in cybersecurity education because it demonstrates the relationship between input handling, application logic, database syntax, authentication, and system trust boundaries. A practical laboratory makes these relationships easier to understand than static examples alone.

1.2 Project Motivation

The project was developed to provide an understandable and visually guided environment in which students can:

- configure a local target URL and select a predefined payload;
- observe DNS resolution, TCP connectivity, HTTP execution, and response analysis;
- compare different SQL injection techniques;
- understand the difference between vulnerable query construction and parameterized queries;
- study attack indicators without targeting public systems;
- present each attack scenario during a cybersecurity course demonstration.

1.3 Project Objectives

The primary objectives are:

1. Build a modular web interface for SQL injection education.
2. Implement six clearly separated attack demonstrations.

3. Execute controlled HTTP-based checks against a local vulnerable application.
4. Display realistic terminal-style progress and result information.
5. Store test history locally in the browser for later review.
6. demonstrate secure authentication through parameterized SQLite queries.
7. provide Docker-based setup for repeatable execution.
8. prevent scanning of arbitrary public Internet targets.
9. ensure the destructive demonstration remains non-destructive.

1.4 Scope

The project focuses on SQL injection awareness and demonstration. It is not a full penetration-testing framework and is not intended to replace professional scanners. The implemented scanner modules test one supplied payload at a time and interpret application-specific response signals. The vulnerable target application is expected to run locally, normally on port 2000, while the educational frontend and Flask attack API run on ports 3000 and 5000 respectively.

2 Requirements Analysis

2.1 Functional Requirements

Table 1: Functional requirements

ID	Requirement
FR-01	The system shall provide a login/entry page and navigation to the main modules.
FR-02	The system shall provide pages for Home, Instructions, Attacks, Payload Library, and Logs.
FR-03	The system shall provide interactive modules for classic, login bypass, UNION, blind, comment, and DROP TABLE attack scenarios.
FR-04	The user shall be able to enter a target URL and SQL payload.
FR-05	The system shall provide predefined local-lab presets.
FR-06	The frontend shall send attack requests to the Flask API.

ID	Requirement
FR-07	The backend shall validate required parameters and return JSON responses.
FR-08	The attack modules shall report network and HTTP execution stages.
FR-09	The frontend shall present scanner output in a terminal-style component.
FR-10	The application shall store test results in browser local storage.
FR-11	The application shall provide a dark/light theme switch.
FR-12	The application shall support responsive navigation and PWA installation.
FR-13	The DROP TABLE module shall demonstrate risk without executing the destructive statement.

2.2 Non-Functional Requirements

Table 2: Non-functional requirements

ID	Requirement
NFR-01	The system shall be understandable for classroom demonstrations.
NFR-02	The codebase shall separate presentation, API routing, database access, and attack logic.
NFR-03	Public Internet targets shall be rejected by the scanner modules.
NFR-04	Network operations shall use timeouts and controlled error handling.
NFR-05	The application shall run consistently through Docker Compose.
NFR-06	The frontend shall use lazy loading to reduce the initial bundle size.
NFR-07	The interface shall remain usable on desktop and mobile screen sizes.
NFR-08	Destructive SQL shall not be executed by the educational DROP TABLE module.
NFR-09	Secure examples shall use parameterized database queries.

3 Technology Stack

Table 3: Main technologies used in the project

Layer	Technology	Purpose
Frontend	React 19	Component-based single-page user interface.
Routing	React Router 7	Hash-based client-side routing and redirects.
Build Tool	Vite 6	Development server, bundling, and optimized builds.
Styling	Tailwind CSS 4	Responsive utility-first styling and theme implementation.
Icons	Lucide React	Consistent interface iconography.
Backend	Flask	HTTP API, blueprint registration, and orchestration of attack modules.
CORS	Flask-Cors	Enables frontend-to-backend requests during local development.
HTTP Client	Requests	Sends HTTP requests from attack modules to the vulnerable target.
Database	SQLite	Stores educational user records for the local backend example.
Containers	Docker / Compose	Reproducible frontend and backend environments.
Testing	Pytest	Available backend testing framework.

4 System Architecture

4.1 Architectural Overview

The project follows a three-part laboratory architecture:

1. **Educational frontend:** a React application on port 3000.
2. **Attack orchestration API:** a Flask service on port 5000.
3. **Vulnerable target:** a separate local Golimar Store application, normally on port 2000.

The browser does not directly implement the attack logic. Instead, it submits the target URL and payload to the Flask API. The backend performs URL parsing, local/private network validation, DNS resolution, connectivity testing, HTTP communication, and response interpretation. It then returns both machine-readable result fields and human-readable terminal output.

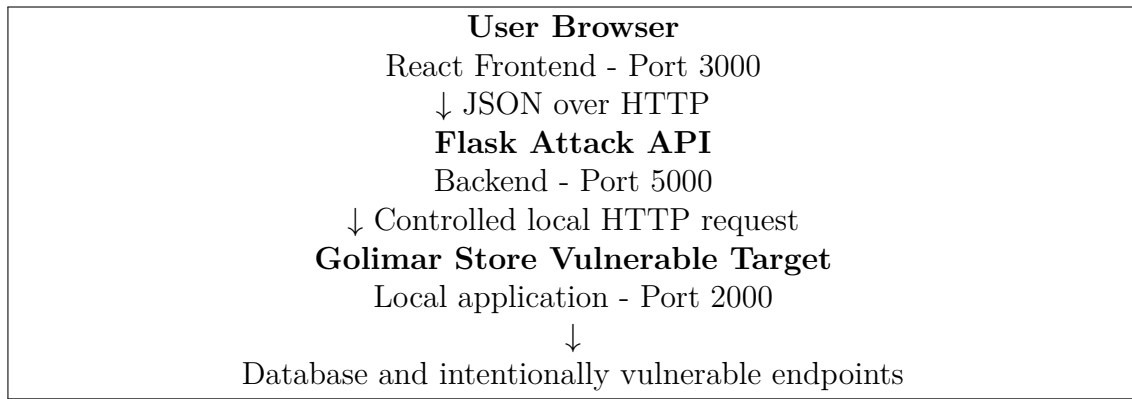


Figure 1: Logical architecture of the SQL injection laboratory

4.2 Frontend Architecture

The frontend is a React single-page application using `HashRouter`. Pages are loaded lazily through `React.lazy` and rendered inside a shared layout. Important frontend responsibilities include:

- route management and backward-compatible redirects;
- target URL and payload input forms;
- predefined attack presets;
- terminal animation and status presentation;
- local log storage;
- responsive desktop/mobile navigation;
- light and dark themes;
- installable PWA behavior.

The primary routes are shown in Table 4.

Table 4: Important frontend routes

Route	Purpose
/ and /login	Entry/login page.
/home	Project overview dashboard.
/instructions	Educational instructions and attack explanations.

Route	Purpose
/attacks	Attack module overview.
/payload-library	Payload reference and categorized examples.
/logs	Locally stored test history.
/attacks/classic-injection	Remote classic SQL injection scanner.
/attacks/login-bypass	Authentication bypass demonstration.
/attacks/union-attack	UNION extraction and table enumeration.
/attacks/blind-injection	Boolean-based blind SQL injection.
/attacks/drop-table	Safe destructive-query capability demonstration.
/attacks/comment-attack	SQL comment-based authentication bypass.

4.3 Backend Architecture

The Flask backend uses the application factory pattern. The `create_app()` function loads configuration, enables CORS, and registers four blueprints:

- `/api`: health and demonstration endpoints;
- `/api/auth`: placeholder login/logout endpoints;
- `/api`: vulnerable and secure demonstration endpoints;
- `/api/attacks`: attack execution endpoints.

Each attack implementation is separated into its own Python module under `backend/attacks`. This keeps routing logic small and allows each scanner to maintain attack-specific parsing and result fields.

4.4 Database Layer

The included backend database is SQLite. The database path is calculated relative to the backend directory and opened with `sqlite3.Row` as the row factory. The schema contains a simple `users` table with `id`, `username`, `password`, and `role` fields.

The secure login example uses placeholders:

```

1 user = connection.execute(
2     """
3     SELECT id, username, role
4     FROM users

```

```
5 WHERE username = ? AND password = ?
6     "",
7     (username, password),
8 ).fetchone()
```

Listing 1: Parameterized secure login query

Because the SQL structure and user data are sent separately, injected SQL operators are treated as data rather than executable syntax.

4.5 Docker Networking

The backend container listens on 0.0.0.0:5000. When an attack target contains `localhost` or `127.0.0.1`, the attack modules detect container execution and translate the hostname to `host.docker.internal`. The Compose configuration adds the following mapping:

```
1 extra_hosts:
2   - "host.docker.internal:host-gateway"
```

Listing 2: Docker host mapping used by the backend

This allows the backend container to reach a target service running on the host machine while preserving the user-facing local URL.

5 Project Structure

```
1 SQL Inject/
2 |-- backend/
3 |   |-- app/
4 |   |   |-- controllers/
5 |   |   |-- models/
6 |   |   |-- __init__.py
7 |   |-- attacks/
8 |   |   |-- blind_injector.py
9 |   |   |-- classic_injector.py
10 |   |   |-- comment_injector.py
11 |   |   |-- droptable.py
12 |   |   |-- login_bypass_injector.py
13 |   |   |-- table_enumeration_injector.py
14 |   |   |-- union_injector.py
15 |   |-- database/
16 |   |   |-- app.db
```

```
17 | | |-- schema.sql
18 | | '-- seed.py
19 | |-- routes/attack_routes.py
20 | |-- config.py
21 | |-- requirements.txt
22 | '-- run.py
23 |-- frontend/
24 | |-- public/
25 | |-- src/
26 | | |-- components/
27 | | |-- layouts/
28 | | |-- pages/
29 | | |-- services/
30 | | '-- utils/
31 | |-- package.json
32 | '-- vite.config.js
33 |-- docs/
34 | |-- main.tex
35 | |-- sql-inject.pptx
36 | '-- sql-injection-attack.pdf
37 |-- screenshots/
38 |-- docker-compose.yml
39 '-- Dockerfile
```

Listing 3: Simplified repository structure

5.1 Frontend Components

- `App.jsx`: application routes, lazy loading, and redirects.
- `Layout.jsx`: navigation, theme switching, PWA installation, mobile menu, footer.
- `Terminal.jsx`: terminal-style output visualization.
- `PageTransition.jsx`: page transition wrapper.
- `ErrorBoundary.jsx`: catches rendering failures.
- `services/api.js`: communication with the Flask backend.
- `utils/terminalAnimation.js`: animated line-by-line scanner output.
- `utils/logger.js`: browser-side test history.

5.2 Backend Modules

- `app/__init__.py`: Flask application factory and blueprint registration.
- `controllers/api\_controller.py`: health endpoint.
- `controllers/demo\_controller.py`: vulnerable/secure demonstration endpoints.
- `models/db.py`: SQLite connection management.
- `models/user\_model.py`: secure parameterized login query.
- `routes/attack\_routes.py`: validates requests and invokes attack modules.
- `attacks/*.py`: network checks, payload transmission, response analysis, and formatted output.

6 API Design

6.1 General Endpoints

Table 5: General backend API endpoints

Method	Endpoint	Purpose
GET	<code>/api/health</code>	Returns API service status.
POST	<code>/api/auth/login</code>	Authentication placeholder endpoint.
POST	<code>/api/auth/logout</code>	Logout placeholder endpoint.
POST	<code>/api/secure/login</code>	Demonstrates parameterized SQLite authentication.
POST	<code>/api/vulnerable/login</code>	Placeholder vulnerable login endpoint.
POST	<code>/api/vulnerable/search</code>	Placeholder vulnerable search endpoint.

6.2 Attack Endpoints

Table 6: Attack execution API endpoints

Method	Endpoint	Main result indicators
POST	<code>/api/attacks/classic</code>	Reachability, request completion, vulnerability detection, response metrics.

Method	Endpoint	Main result indicators
POST	/api/attacks/login-bypass	Reachability, HTTP completion, authentication bypass, final URL.
POST	/api/attacks/union	UNION leak detection, leaked rows, executed query, response metrics.
POST	/api/attacks/table-enumeration	Enumerated table names, executed query, table count.
POST	/api/attacks/blind	Boolean condition result, response length, elapsed time.
POST	/api/attacks/comment	Comment injection result, redirect state, cookies, final URL.
POST	/api/attacks/drop-table	Vulnerability capability, stacked-query indicator, proof that no destructive execution occurred.

6.3 Request Format

Most attack routes expect JSON containing `target_url` and `payload`:

```

1 {
2   "target_url": "http://127.0.0.1:2000/api/vulnerable/search",
3   "payload": "' UNION SELECT ... -- "
4 }
```

Listing 4: Typical attack request body

The comment attack may additionally include a `password` value. Missing parameters produce an HTTP 400 response. Internal failures produce an HTTP 500 response with an error message.

7 User Interface

7.1 Home Dashboard

The home page introduces the laboratory, summarizes the available modules, and provides direct navigation to the educational workflow.

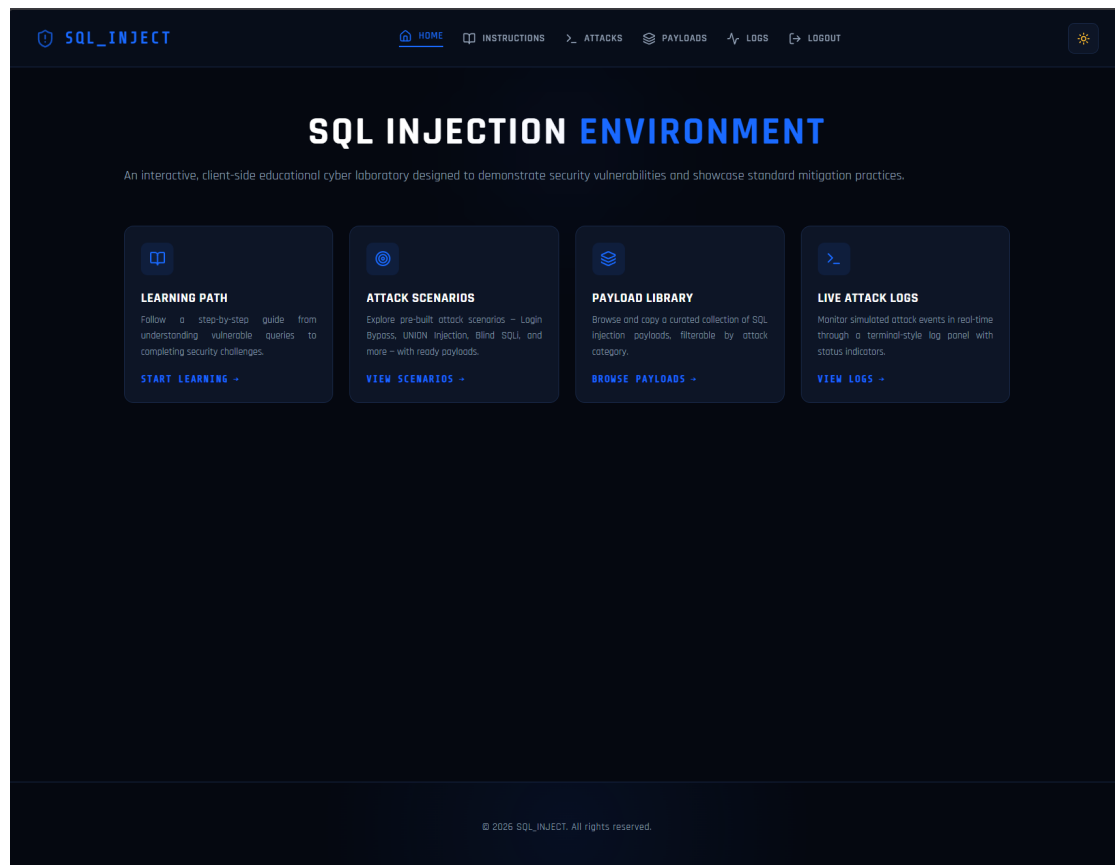


Figure 2: Home dashboard of the SQL injection laboratory

7.2 Instructions

The instructions module explains the theory, purpose, payload structure, expected behavior, and mitigation of each attack category.

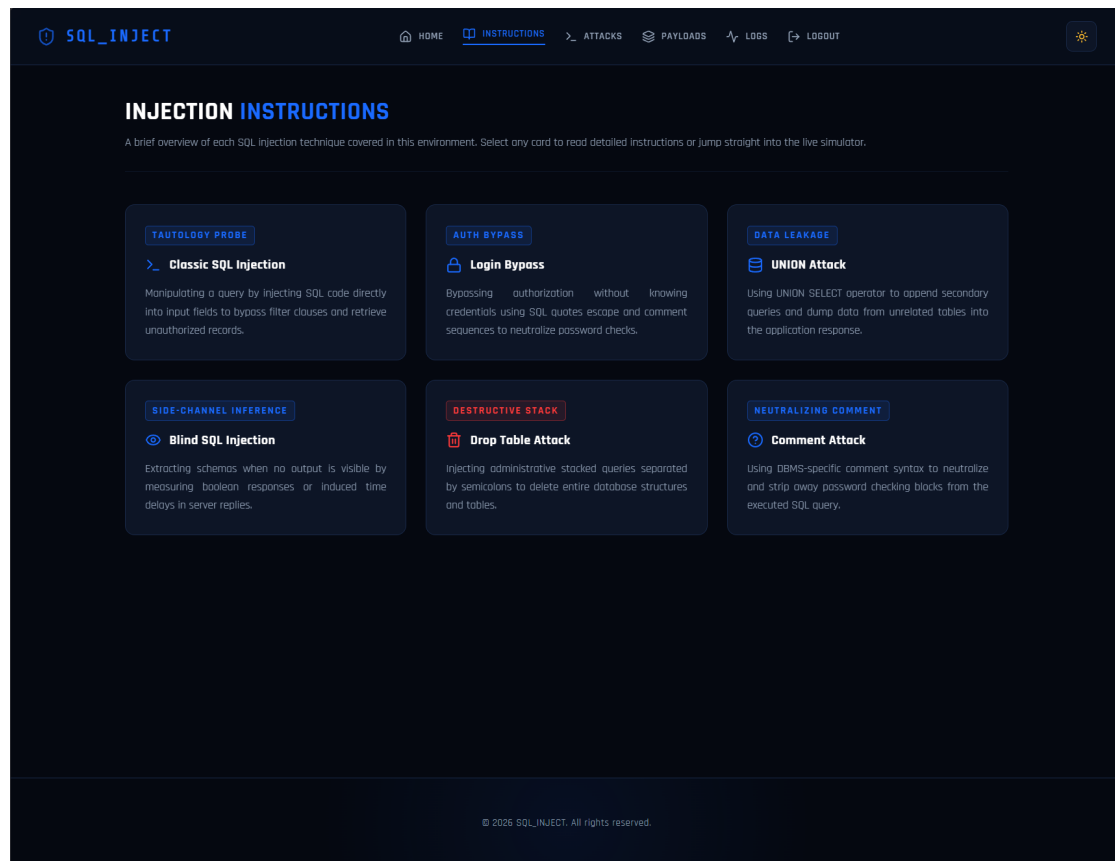


Figure 3: Instructions overview

7.3 Attack Overview

The attack overview presents the six supported techniques as selectable cards. Each card opens a dedicated remote scanner page.

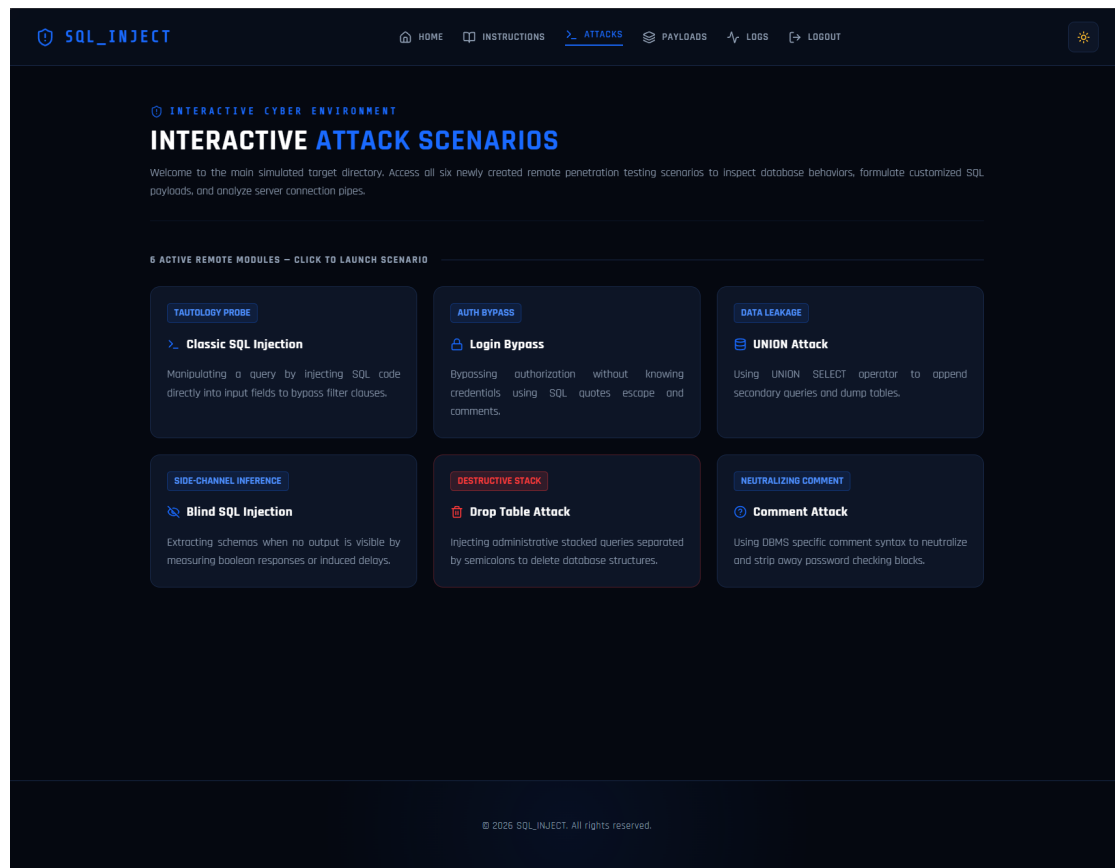


Figure 4: Attack module overview

7.4 Payload Library

The payload library is a reference section containing attack-specific payload examples. Its purpose is to support learning and simplify controlled demonstrations.

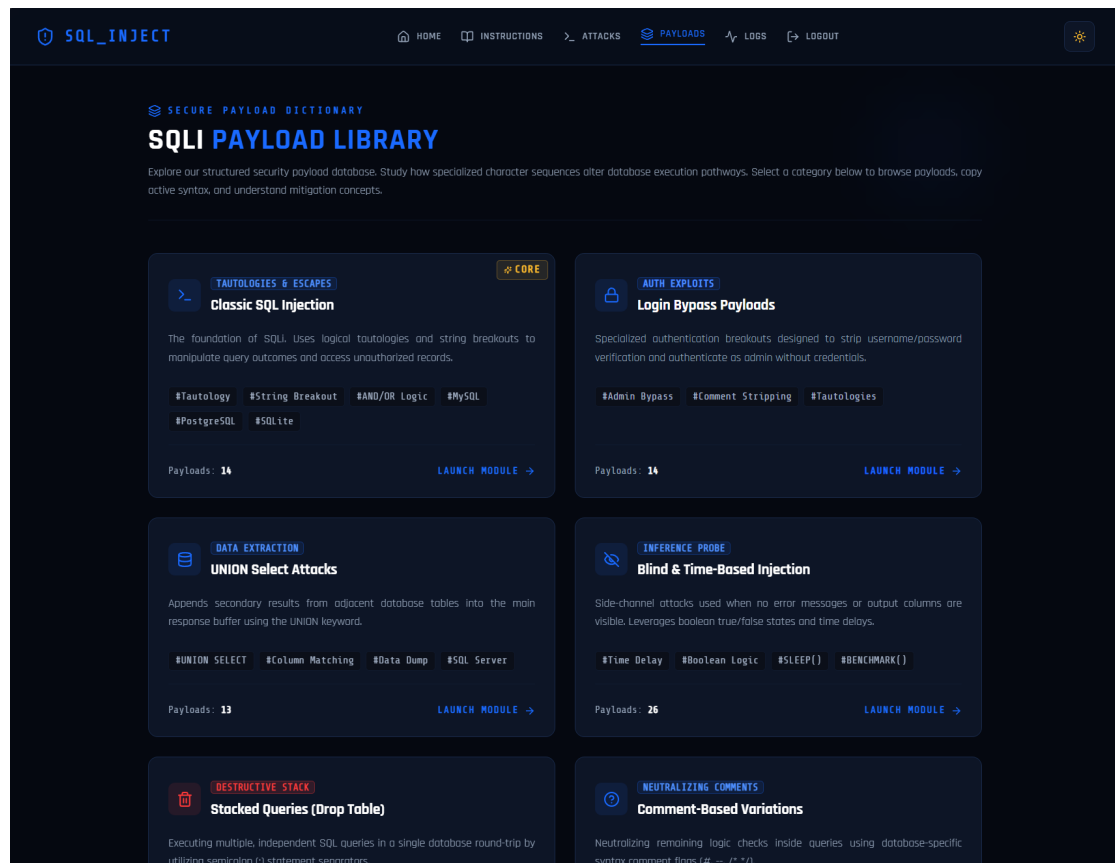
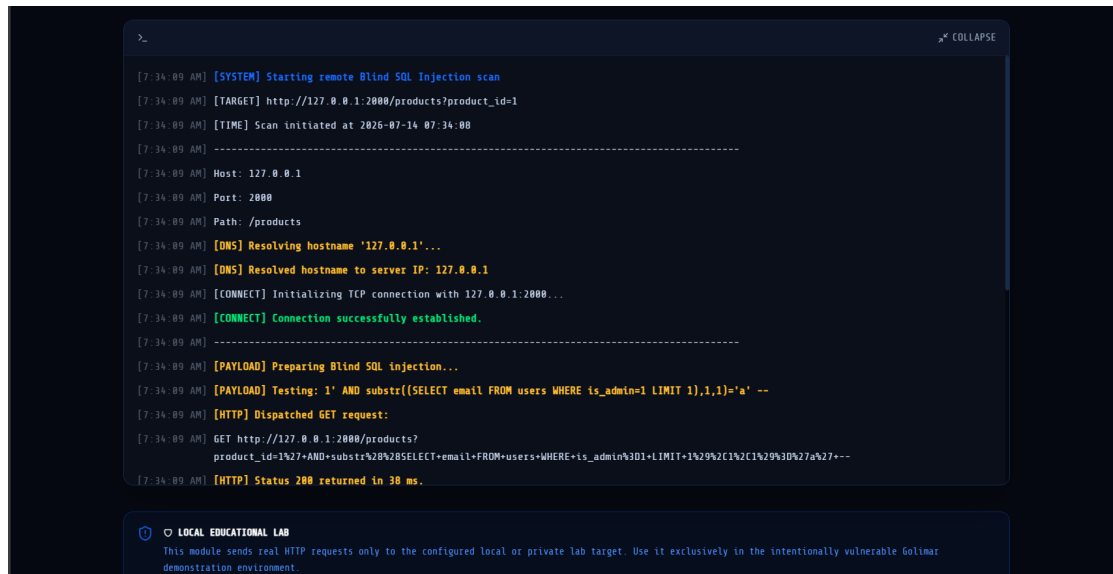


Figure 5: Payload library

7.5 Terminal Output

Each interactive attack page contains a terminal component. It displays scanner stages such as system initialization, target parsing, DNS resolution, connection setup, payload handling, HTTP response analysis, and final result classification.



```
[7:34:09 AM] [SYSTEM] Starting remote Blind SQL Injection scan
[7:34:09 AM] [TARGET] http://127.0.0.1:2000/products?product_id=1
[7:34:09 AM] [TIME] Scan initiated at 2026-07-14 07:34:00
[7:34:09 AM] -----
[7:34:09 AM] Host: 127.0.0.1
[7:34:09 AM] Port: 2000
[7:34:09 AM] Path: /products
[7:34:09 AM] [DNS] Resolving hostname '127.0.0.1'...
[7:34:09 AM] [DNS] Resolved hostname to server IP: 127.0.0.1
[7:34:09 AM] [CONNECT] Initializing TCP connection with 127.0.0.1:2000...
[7:34:09 AM] [CONNECT] Connection successfully established.
[7:34:09 AM] -----
[7:34:09 AM] [PAYLOAD] Preparing Blind SQL Injection...
[7:34:09 AM] [PAYLOAD] Testing: 1' AND substr((SELECT email FROM users WHERE is_admin=1 LIMIT 1),1,1)='a' --
[7:34:09 AM] [HTTP] Dispatched GET request:
[7:34:09 AM] GET http://127.0.0.1:2000/products?product_id=1&27+AND+substr%28%28SELECT+email+FROM+users+WHERE+is_admin%3D1+LIMIT+1%29%2C1%29%3D%27a%27+--+
[7:34:09 AM] [HTTP] Status 200 returned in 38 ms.
```

LOCAL EDUCATIONAL LAB
This module sends real HTTP requests only to the configured local or private lab target. Use it exclusively in the intentionally vulnerable Golimar demonstration environment.

Figure 6: Terminal-style scanner output

8 Attack Modules

8.1 Common Attack Workflow

Although each attack module has specialized detection logic, the general workflow is consistent:

1. Read the target URL and payload from the frontend.
2. Validate that both fields are present.
3. Parse the scheme, hostname, port, path, and query parameters.
4. Translate loopback addresses when running inside Docker.
5. Resolve the hostname through DNS.
6. verify that the resolved address is loopback or private.
7. Open a short TCP connection to verify reachability.
8. Send the attack-specific HTTP request when permitted.
9. Measure status code, elapsed time, response length, redirection, and content signals.
10. Return structured JSON and formatted terminal output.
11. Save the summarized result in browser local storage.

8.2 Classic SQL Injection

8.2.1 Concept

Classic SQL injection modifies a query by closing the original quoted value and adding an always-true condition. For example, a vulnerable query may be constructed as follows:

```
1 SELECT * FROM products
2 WHERE category = '<USER_INPUT>';
```

Listing 5: Conceptual vulnerable search query

Supplying a tautology such as `1' OR '1'='1` can change the WHERE clause so that it matches more rows than intended.

8.2.2 Project Implementation

The classic scanner supports editable target URLs, a payload catalog, and multiple presets. Example presets include product category, customer search, and public book database patterns. The backend performs a real HTTP request to the local target and classifies the result using response-specific evidence.

8.2.3 Security Impact

A successful classic injection may expose unauthorized records, bypass filters, or change application behavior. The primary defense is a parameterized query combined with strict server-side validation.

8.3 Login Bypass

8.3.1 Concept

Login bypass occurs when injected input changes an authentication query so that the password condition becomes irrelevant. A simplified vulnerable pattern is:

```
1 SELECT * FROM users
2 WHERE email = '<EMAIL>'
3 AND password = '<PASSWORD>';
```

Listing 6: Conceptual vulnerable authentication query

The project preset uses:

```
1 admin ' OR 1=1 --
```

Listing 7: Login bypass payload used by the project

The quote terminates the original value, `OR 1=1` creates a true expression, and `-` comments out the remaining password check.

8.3.2 Demonstration Target

- Target: `http://127.0.0.1:2000/login`
- Preset: Golimar Store Login
- Result field: `authentication\_bypassed`

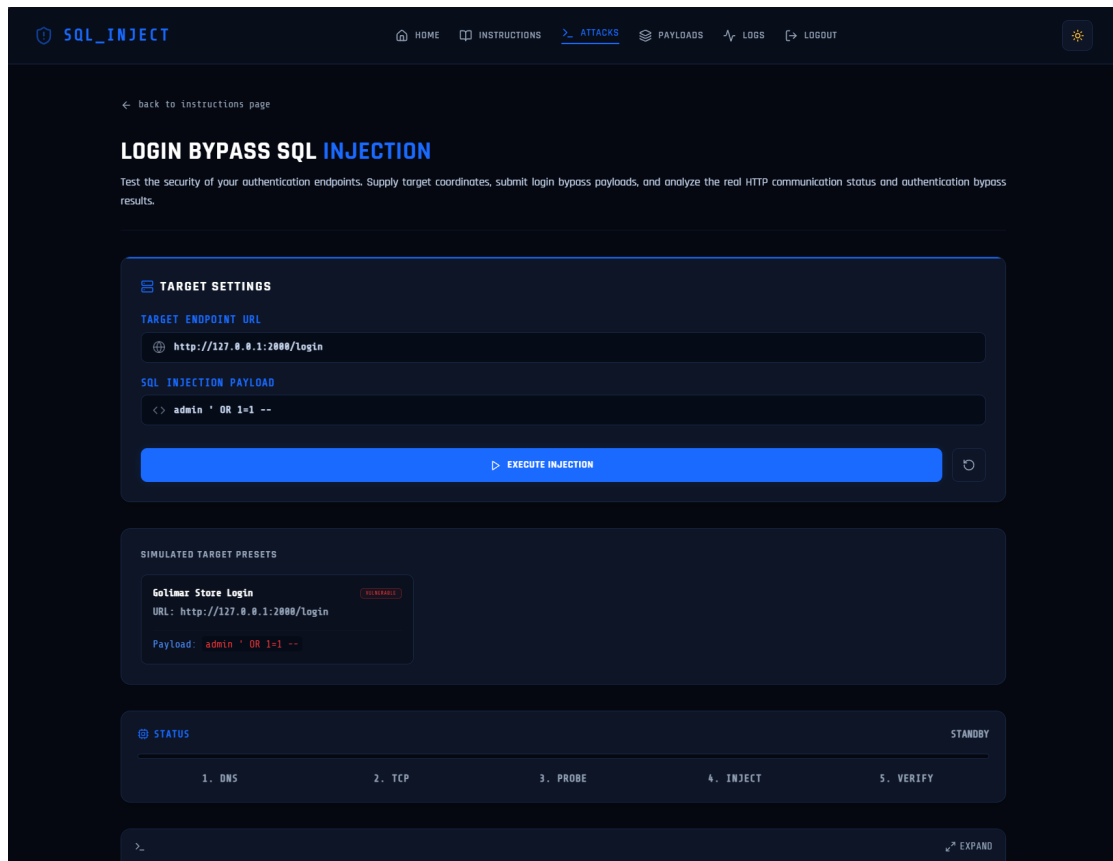


Figure 7: Remote login bypass module

8.4 UNION-Based SQL Injection

8.4.1 Concept

The SQL UNION operator combines result sets from compatible SELECT statements. If an injectable query returns a known number of columns with compatible data types, a crafted UNION clause can retrieve data from another table.

The main project payload is:

```
1 ' UNION SELECT id,email,0,is_admin,'','password','user-record'  
2 FROM users --
```

Listing 8: UNION payload for retrieving user records

The scanner sends the payload to the local vulnerable search endpoint and analyzes the JSON or HTML response for leaked rows.

8.4.2 Table Enumeration

The UNION page also includes a second preset for database table-name enumeration:

```
1 ' UNION SELECT ROW_NUMBER() OVER ( ),table_name,0,0,  
2 'Database table discovered through information_schema','',  
3 'table-record'  
4 FROM information_schema.tables  
5 WHERE table_schema='public' --
```

Listing 9: Table enumeration payload

This preset uses the dedicated `/api/attacks/table-enumeration` backend route and returns the discovered table names and count.

8.4.3 Demonstration Target

- Target: `http://127.0.0.1:2000/api/vulnerable/search`
- Modes: user-record extraction and table enumeration
- Key results: leaked rows, table names, executed query, response metrics

8.5 Boolean-Based Blind SQL Injection

8.5.1 Concept

Blind SQL injection is used when database values are not directly displayed. Instead, the attacker asks true/false questions and infers information from observable differences such

as response content, status, timing, or page length.

The project uses a Boolean condition similar to:

```
1 1' AND substr(  
2   (SELECT email FROM users WHERE is_admin=1 LIMIT 1),  
3   1,1  
4 )='a' --
```

Listing 10: Boolean-based blind payload

This asks whether the first character of the selected administrator email is a.

8.5.2 Demonstration Target

- Target: `http://127.0.0.1:2000/products?product\_id=1`
- Injected parameter: `product\_id`
- Main result: `blind\_condition`

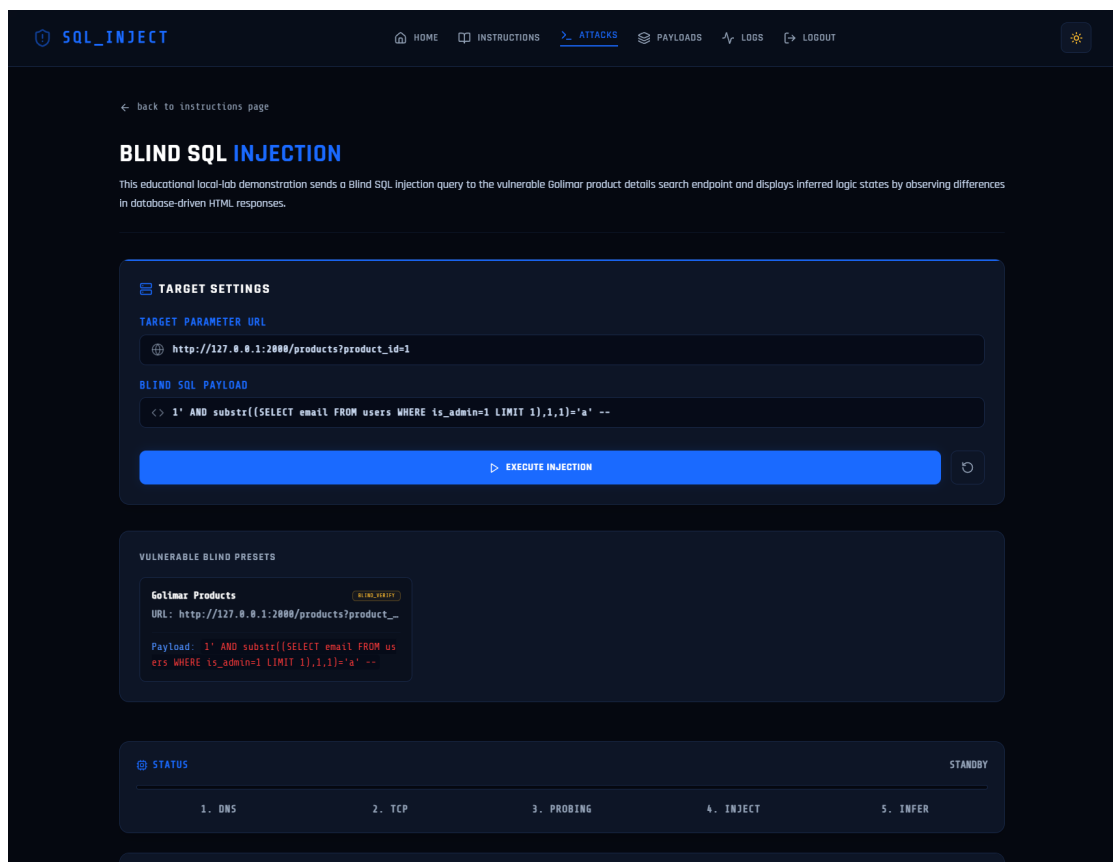


Figure 8: Boolean-based blind SQL injection module

8.6 SQL Comment Attack

8.6.1 Concept

SQL comment tokens can remove the remaining part of a vulnerable statement. In authentication queries, this can eliminate the password condition after a valid or guessed account identifier.

The project preset uses:

```
1 admin@example.com' --
```

Listing 11: Comment injection payload

The backend submits the payload and a test password, then evaluates indicators such as HTTP status, redirection, cookies, response length, and final URL.

8.6.2 Demonstration Target

- Target: `http://127.0.0.1:2000/login`
- Password: `test`
- Main result: `comment\_injection`

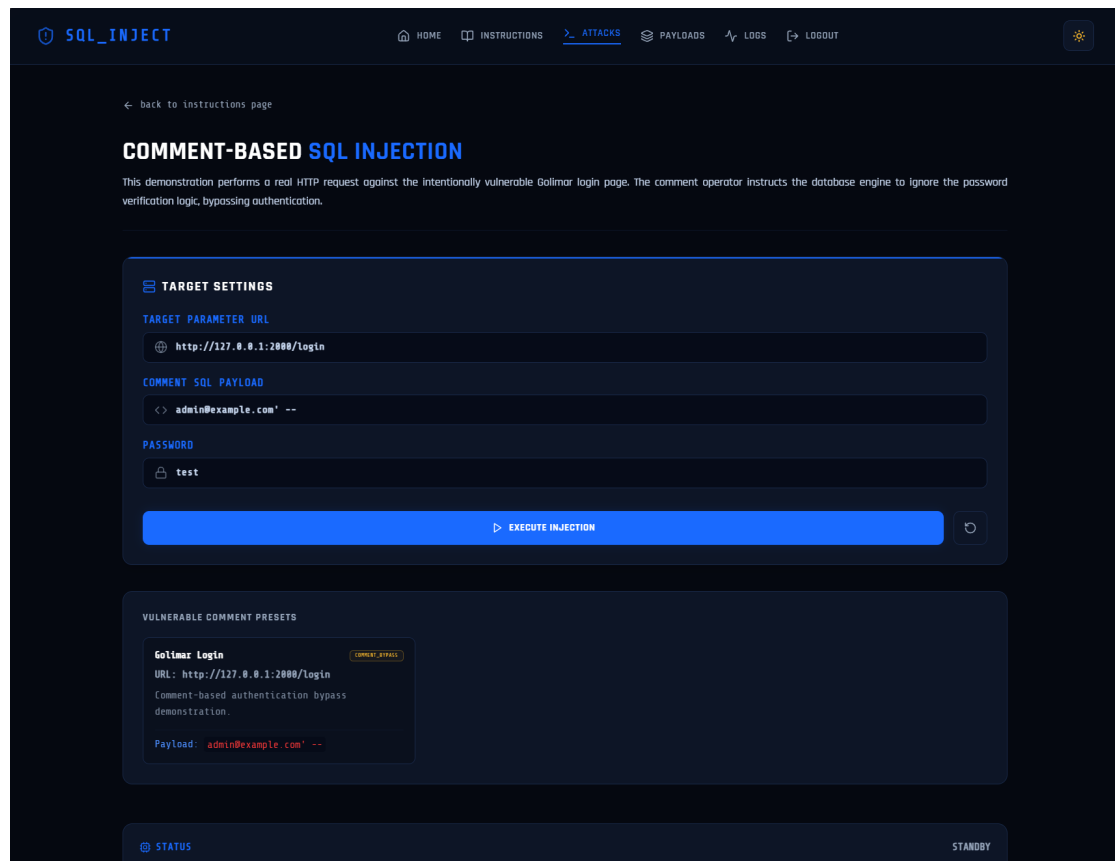


Figure 9: SQL comment attack module

8.7 DROP TABLE Demonstration

8.7.1 Concept

A stacked-query vulnerability may allow an attacker to terminate the original statement and append another SQL command. The educational payload displayed by the project is:

```
1 ' ; DROP TABLE products; --
```

Listing 12: Displayed destructive payload

If such a payload were executed against a vulnerable database driver that permits stacked statements, the `products` table could be removed.

8.7.2 Non-Destructive Safety Design

The final project implementation intentionally does not transmit or execute the destructive statement. The frontend explicitly informs the user that the payload is displayed only for educational purposes. The module performs a safe capability assessment and verifies that the table remains available.

The result model includes:

- vulnerability_detected;
- stacked_queries_possible;
- destructive_statement_executed, expected to remain false;
- table_exists_before;
- table_exists_after.

This design preserves the teaching value while preventing accidental data loss.

8.7.3 Demonstration Target

- Target: `http://127.0.0.1:2000/api/vulnerable/drop-products`
- Payload status: displayed but not dispatched
- Expected result: vulnerability indication with the product table preserved

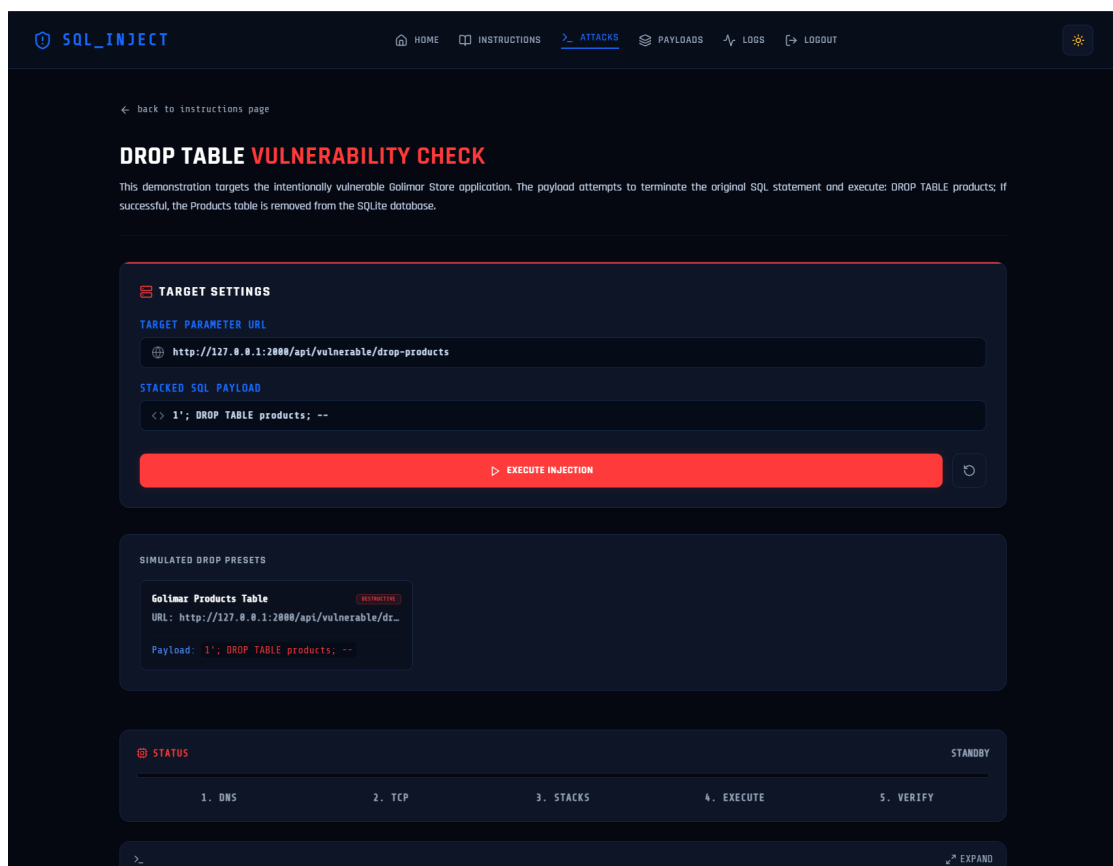


Figure 10: Non-destructive DROP TABLE vulnerability demonstration

9 Safety and Ethical Controls

9.1 Private-Network Restriction

The attack modules resolve the supplied hostname and inspect the resulting IP address. Requests are allowed only when the address is loopback or private, or when Docker host translation is active. Public Internet targets are rejected with a safety error.

This restriction reduces the possibility of using the project against unauthorized systems. It does not replace legal and ethical responsibility; users must still have explicit permission for every target.

9.2 Controlled Connectivity

Before sending an HTTP request, the backend performs a TCP connectivity test with a short timeout. The request itself also uses timeouts and exception handling. Failures are returned as structured results rather than causing the API process to terminate.

9.3 Destructive Payload Handling

The DROP TABLE page is intentionally non-destructive. The payload is visible so students can analyze its syntax, but the module does not dispatch it. This is an important distinction between demonstrating a vulnerability and damaging the target environment.

9.4 Data Handling

Test logs are stored in the browser's local storage. They should be treated as local demonstration data and should not contain credentials or sensitive production information. The bundled SQLite database includes only artificial educational accounts.

10 Secure Implementation Principles

10.1 Parameterized Queries

The most important mitigation is to use prepared or parameterized statements. The secure login model in this project demonstrates the correct SQLite pattern with ? placeholders. User values are bound separately from the SQL command.

10.2 Password Security

The educational SQLite schema stores plaintext passwords to keep the demonstration simple. This must never be copied into a production system. A real application must:

- hash passwords with a modern password-hashing algorithm such as Argon2id or bcrypt;
- generate a unique salt automatically through the hashing library;
- use constant-time verification functions;
- enforce rate limiting and account protection;
- use secure session cookies and CSRF protection.

10.3 Least Privilege

The database account used by an application should have only the permissions necessary for its normal operation. A search endpoint should not connect with a role capable of dropping tables or managing database users.

10.4 Input Validation

Input validation supports, but does not replace, parameterization. Examples include validating numeric identifiers as integers, applying maximum lengths, using allow-lists for sortable columns, and rejecting unexpected request structures.

10.5 Error Handling

Database errors should be logged securely on the server and translated into generic client responses. Returning raw SQL errors can reveal table names, column names, query structure, or database technology.

10.6 Security Comparison

Table 7: Vulnerable and secure query construction

Aspect	Vulnerable approach	Secure approach
Query creation	String concatenation	Parameterized statement
Input meaning	May become SQL syntax	Always treated as data
Error output	Raw database errors	Generic client errors
Database role	Excessive privileges	Least-privileged account
Passwords	Plaintext comparison	Strong password hashing
Monitoring	No security logging	Authentication and anomaly logging

11 Installation and Execution

11.1 Prerequisites

The recommended environment requires:

- Docker Desktop or Docker Engine with Docker Compose;
- a modern web browser;
- the separate Golimar Store vulnerable target running locally when demonstrating real attack requests;
- optionally Node.js 20 and Python 3.11 for non-container development.

11.2 Docker Compose Execution

From the repository root, run:

```
1 docker compose up --build
```

Listing 13: Starting the project with Docker Compose

After startup:

- Frontend: <http://localhost:3000>
- Backend health endpoint: <http://localhost:5000/api/health>
- Vulnerable target: normally <http://127.0.0.1:2000>

To stop the containers:

```
1 docker compose down
```

Listing 14: Stopping the project

11.3 Manual Backend Execution

```
1 cd backend
2 python -m venv .venv
3 source .venv/bin/activate      # Linux/macOS
4 # .venv\Scripts\activate      # Windows PowerShell
5 pip install -r requirements.txt
6 python run.py
```

Listing 15: Running the Flask backend manually

11.4 Manual Frontend Execution

```
1 cd frontend
2 npm install
3 npm run dev
```

Listing 16: Running the React frontend manually

11.5 Database Initialization

The included SQLite database can be recreated by executing:

```
1 cd backend
2 python database/seed.py
```

Listing 17: Recreating the educational SQLite database

The script reads `database/schema.sql`, recreates the `users` table, inserts demonstration accounts, and commits the database.

12 Demonstration Procedure

A recommended classroom demonstration follows these steps:

1. Start the Golimar Store vulnerable application on port 2000.
2. Start the SQL_INJECT frontend and backend through Docker Compose.
3. Open <http://localhost:3000> and enter the application.
4. Present the Instructions page and explain SQL query construction.
5. Open the Attacks page and select one attack category.
6. Select the corresponding Golimar preset.
7. Explain the target URL and payload before execution.
8. Execute the scan and follow the terminal stages.
9. Explain the final result and the evidence used by the scanner.
10. Open the Logs page to show the recorded test result.
11. Present the secure alternative using parameterized queries.

12.1 Example: Login Bypass Presentation

1. Explain that the login endpoint combines email and password conditions.
2. Show the payload `admin ' OR 1=1 --`.
3. Identify the three parts: quote termination, tautology, and comment token.
4. Execute the preset against the local login page.
5. Observe DNS, connection, HTTP request, redirection, and authentication result.
6. Conclude with the parameterized query mitigation.

13 Testing Strategy

13.1 Backend Unit and Integration Tests

Recommended backend tests include:

- application factory creates a Flask application successfully;
- `/api/health` returns HTTP 200 and expected JSON;

- missing attack parameters return HTTP 400;
- public IP targets are rejected;
- malformed URLs produce controlled error results;
- unreachable local ports produce a failed connectivity result;
- secure login accepts valid demonstration credentials;
- secure login rejects injected input;
- DROP TABLE result always reports no destructive execution;
- table existence remains unchanged after the DROP TABLE demonstration.

13.2 Frontend Tests

Recommended frontend tests include:

- all main routes render without errors;
- old attack URLs redirect to the new `/attacks/...` paths;
- preset selection updates target URL and payload;
- empty target or payload is rejected before execution;
- loading and result states are displayed correctly;
- terminal expansion does not exceed the page width;
- theme preference persists in local storage;
- mobile navigation opens and closes correctly;
- logs are saved and rendered correctly;
- PWA manifest and service worker files are available.

13.3 Manual Acceptance Test Matrix

Table 8: Manual acceptance test matrix

ID	Test	Expected result	Status
T-01	Open frontend	Login/entry page loads.	To be verified
T-02	Backend health check	JSON status reports success.	To be verified
T-03	Classic local preset	Terminal completes and result is classified.	To be verified
T-04	Login bypass preset	Authentication bypass evidence is displayed.	To be verified
T-05	UNION user extraction	Leaked rows are displayed when target is vulnerable.	To be verified
T-06	Table enumeration	Discovered table names are listed.	To be verified
T-07	Blind true/false condition	Boolean result reflects target response.	To be verified
T-08	Comment attack	Redirect/cookie/response indicators are shown.	To be verified
T-09	DROP TABLE demo	No destructive statement is transmitted or executed.	To be verified
T-10	Public target URL	Backend rejects the target.	To be verified
T-11	Logs page	Completed test appears in local history.	To be verified
T-12	Mobile layout	Navigation and attack forms remain usable.	To be verified

14 Known Limitations

- Detection logic is tailored to the local Golimar Store responses and may not generalize to other applications.
- Most scans test a single payload rather than performing automated payload generation.
- The bundled vulnerable and secure demonstration endpoints are partly placeholders.
- The included SQLite user database stores educational plaintext passwords and is not production-ready.
- Browser logs are local to one device and are not centrally managed.

- The backend runs with Flask debug mode enabled in the current development configuration.
- CORS is enabled broadly for local development.
- No authentication or authorization protects the attack API itself.
- Automated test coverage is not yet included in the submitted repository.
- The attack service depends on a separately running vulnerable target for complete demonstrations.

15 Recommended Improvements

1. Add a complete automated Pytest suite and frontend component tests.
2. Replace placeholder vulnerable/secure endpoints with isolated, resettable lab scenarios.
3. Add a one-click database reset function for repeatable demonstrations.
4. Disable Flask debug mode outside local development.
5. Restrict CORS to the known frontend origin.
6. Add authentication or a local-only binding policy for the attack API.
7. Add structured server-side logging with request correlation identifiers.
8. Add timing graphs for blind SQL injection comparisons.
9. Add side-by-side vulnerable and secure source-code explanations.
10. Add a lab status page that checks frontend, backend, target, and database readiness.
11. Remove generated directories such as `node_modules`, `dist`, and `__pycache__` from version control.
12. Add a repository-level `.gitignore`, license, contribution guide, and security policy.

16 Team Organization

The project was completed as a Group C cybersecurity course project. Work was divided across project coordination, frontend implementation, backend/API development, database preparation, attack research, testing, presentation preparation, and documentation. Git-based collaboration allowed features to be developed independently and integrated into the final demonstration environment.

A recommended responsibility model is shown below. It may be adjusted to match the final officially agreed team allocation.

Table 9: Project responsibility areas

Responsibility	Main tasks
Project coordination	Scope, integration planning, final review, and presentation coordination.
Frontend development	React pages, routes, components, responsive layout, themes, and terminal UI.
Backend development	Flask application, API routes, validation, and error handling.
Attack implementation	HTTP scanner modules, payload handling, and response analysis.
Database engineering	SQLite schema, seed data, and secure query examples.
Testing and quality assurance	Functional checks, safety verification, Docker validation, and bug reporting.
Documentation and presentation	LaTeX report, screenshots, diagrams, slides, and speaking material.

17 Conclusion

The SQL_INJECT project provides a practical and visually structured environment for learning SQL injection. Its strongest aspect is the integration of theory, payload configuration, real local HTTP communication, terminal-style feedback, and secure coding guidance in one application. The separation between React presentation logic, Flask routing, attack-specific Python modules, and SQLite data access makes the system understandable and extensible.

The project demonstrates six important SQL injection patterns: tautology injection, login bypass, UNION-based extraction, table enumeration, Boolean-based blind inference, comment-based bypass, and stacked-query risk. At the same time, it includes essential safety controls. Scanning is restricted to local or private targets, and the DROP TABLE scenario is explicitly non-destructive.

For production-quality security tooling, further work would be needed in authentication,

test coverage, configuration hardening, generalized detection, and secure secret management. However, as a controlled cybersecurity course laboratory, the project successfully demonstrates why unsafe SQL construction is dangerous and why parameterized queries, least privilege, secure password storage, validation, and careful error handling are necessary.

A Selected Configuration Files

A.1 Docker Compose Configuration

```
1 version: '3.8'
2
3 services:
4   backend:
5     build:
6       context: ./backend
7       dockerfile: Dockerfile
8     ports:
9       - "5000:5000"
10    environment:
11      - FLASK_APP=run.py
12      - FLASK_ENV=development
13    volumes:
14      - ./backend:/app
15    extra_hosts:
16      - "host.docker.internal:host-gateway"
17    stdin_open: true
18    tty: true
19
20 frontend:
21   build:
22     context: ./frontend
23     dockerfile: Dockerfile
24   ports:
25     - "3000:3000"
26   environment:
27     - VITE_PORT=3000
28   volumes:
29     - ./frontend:/app
30     - /app/node_modules
31   depends_on:
```

```

32     - backend
33     stdin_open: true
34     tty: true

```

Listing 18: Project Docker Compose configuration

A.2 Flask Application Factory

```

1  from flask import Flask
2  from flask_cors import CORS
3  from config import Config
4
5  def create_app(config_class=Config):
6      app = Flask(__name__)
7      app.config.from_object(config_class)
8      CORS(app)
9
10     from .controllers.api_controller import api_bp
11     from .controllers.auth_controller import auth_bp
12     from .controllers.demo_controller import demo_bp
13     from routes.attack_routes import attack_bp
14
15     app.register_blueprint(api_bp, url_prefix='/api')
16     app.register_blueprint(auth_bp, url_prefix='/api/auth')
17     app.register_blueprint(demo_bp, url_prefix='/api')
18     app.register_blueprint(attack_bp, url_prefix='/api/attacks')
19
20     return app

```

Listing 19: Simplified Flask application factory

B Payload Reference

Table 10: Payloads included in the interactive presets

Attack	Payload
Classic	1' OR '1'='1
Login bypass	admin ' OR 1=1 -

Attack	Payload
UNION extrac- tion	' UNION SELECT id,email,0,is_admin,",password,'user-record' FROM users -
Blind SQLi	1' AND substr((SELECT email FROM users WHERE is_admin=1 LIMIT 1),1,1)='a' -
Comment attack	admin@example.com' -
DROP TABLE	1'; DROP TABLE products; - (display only; not exe- cuted)

C References

References

- [1] OWASP Foundation, *SQL Injection*, OWASP Web Security Testing Guide and OWASP vulnerability documentation.
- [2] OWASP Foundation, *OWASP Top 10: Injection*, Web application security risk guidance.
- [3] Pallets Projects, *Flask Documentation*, Flask web application framework documentation.
- [4] Meta Open Source, *React Documentation*, React user interface library documentation.
- [5] SQLite Consortium, *SQLite Documentation*, SQLite SQL language and parameter binding documentation.
- [6] Docker Inc., *Docker and Docker Compose Documentation*, Container build and multi-service orchestration documentation.